

MANUAL TECNICO

Pruebas_Simulacion

Sistema de Simulacion de Eventos Discretos

Version 1.0 | Java 25.0.1 | Windows x64

Mayo 2026

Autoria

- Herrera Martínez Aylin
herrera.martinez.aylin23@gmail.com
- Gutiérrez Rivas Alejandro
ale.gtzrivas@gmail.com
- Vázquez Alviso Bryan Alexei
bavazqueza@gmail.com

1. Introducción

Pruebas_Simulacion es una aplicación de escritorio desarrollada en Java que modela una línea de producción industrial mediante Simulación de Eventos Discretos (SED). El sistema genera números pseudoaleatorios a través de un Generador Lineal Congruencial (LGC) parametrizable por el usuario y los utiliza para alimentar las distribuciones estadísticas de tres etapas productivas: llegada de piezas, inspección y empaque.

El paquete incluye dos versiones funcionales empaquetadas como ejecutables Windows autocontenidos (no requieren Java instalado en el equipo):

Version	Ejecutable	Descripción
Main	Main.exe	Simulación con un único inspector y un único empacador. Cola de empaque limitada a 5 piezas. Calcula y reporta utilización de recursos, utilidad económica y cuello de botella.
Prueba_2	Prueba_2.exe	Simulación con un único inspector y dos empaques independientes (Empacador1 y Empacador2). Muestra tablas detalladas de cada numero pseudoaleatorio generado. Calcula y reporta utilización de recursos, utilidad económica y cuello de botella.

2. Arquitectura del Sistema

2.1 Estructura de Directorios

El archivo Pruebas.zip se descomprime con la siguiente jerarquía. Ambas versiones son independientes y cada una lleva su propio runtime JVM embebido:

```
Pruebas/
  Main/
    Main.exe          <- Launcher nativo (JPackage)
    app/
      Pruebas_Simulacion.jar  <- Bytecode compartido (ambas versiones)
      Main.cfg               <- Clase principal y opciones JVM
      .jpackage.xml          <- Metadatos de empaquetado
    runtime/              <- JVM embebida (OpenJDK 25.0.1)
      bin/                  <- DLLs del runtime
      lib/                  <- Modulos Java (jlink)
      conf/                 <- Configuracion de seguridad y red
  Prueba_2/
    Prueba_2.exe
    app/
      Pruebas_Simulacion.jar  <- Mismo JAR que Main
      Prueba_2.cfg
    runtime/               <- JVM identica a la de Main
```

2.2 Archivo JAR Compartido

Ambas versiones comparten el mismo archivo Pruebas_Simulacion.jar. La clase que se ejecuta en cada caso esta determinada por el archivo .cfg de cada versión, no por el MANIFEST.MF del JAR:

Version	Clase ejecutada	Paquete Java	Descripción
Main	Main	Main.Main	1 empacador, cola max 5, cuello de botella.
Prueba_2	Prueba_2	Prueba_2.Prueba_2	2 empacadores, cola max 5, imprime tablas RI, cuello de botella.

2.3 Configuración de Arranque (.cfg)

Cada ejecutable lee su .cfg para determinar la clase principal y las opciones de la JVM.
Ejemplo para Main:

```
[Application]
app.classpath=$APPDIR\Pruebas_Simulacion.jar
app.mainclass=Main.Main

[JavaOptions]
java-options=-Djpackage.app-version=1.0
```

Nota: Modificar `app.mainclass` a un valor incorrecto causa error de clase no encontrada al iniciar.

3. Descripción de Clases y Métodos

3.1 Variables de Estado (Ambas Clases)

Ambas clases declaran las siguientes variables estáticas que representan el estado completo de la simulación en cada instante de tiempo:

Variable	Tipo	Valor inicial	Descripcion
TIEMPO_SIMULACION	double (const)	2400.0	Duracion total de la simulacion en minutos (40 horas).
riLlegadas[]	double[]	null	Secuencia de numeros pseudoaleatorios para llegadas.
riInspeccion[]	double[]	null	Secuencia para tiempos de inspeccion.
riEmpaque[]	double[]	null	Secuencia para tiempos de empaque.
indiceLlegadas	int	0	Puntero al siguiente RI de llegadas a consumir.
indiceInspeccion	int	0	Puntero al siguiente RI de inspeccion.
indiceEmpaque	int	0	Puntero al siguiente RI de empaque.
reloj	double	0	Tiempo actual de simulacion.
proximaLlegada	double	-	Tiempo programado de la siguiente llegada.
finInspeccion	double	MAX_VALUE	Tiempo en que termina la inspeccion en curso.
inspectorOcupado	boolean	false	Estado del inspector.
colaInspeccion	Queue<Integer>	vacía	Cola FIFO de piezas esperando inspeccion.
colaEmpaque	Queue<Integer>	vacía	Cola FIFO de piezas esperando empaque.
piezasLlegadas	int	0	Total de piezas que llegaron al sistema.

Variable	Tipo	Valor inicial	Descripcion
piezasInspeccionadas	int	0	Total de piezas que completaron inspeccion.
piezasEmpacadas	int	0	Total de piezas que completaron empaque.
piezasPerdidas	int	0	Piezas rechazadas por cola de empaque llena.
tiempoOcupadoInspector	double	0	Acumulado del tiempo que el inspector estuvo activo.
tiempoOcupadoEmpacador	double	0	Acumulado del tiempo que el empacador estuvo activo.

Diferencia Main: *Prueba_2* agrega *empacador1Ocupado*, *empacador2Ocupado*, *finEmpaque1* y *finEmpaque2* en lugar de un único *finEmpaque* y *empacadorOcupado*.

3.2 generadorLGC()

Solicita por consola los parámetros del LGC, genera la secuencia y la devuelve como array de doubles normalizados en [0, 1).

Entrada	Tipo Java	Descripcion
a	long	Multiplicador del LGC.
c	long	Incremento (constante aditiva).
x0	long	Semilla (valor inicial X0).
m	long	Modulo (divisor de la congruencia).
n	int	Cantidad de numeros a generar.

Formula aplicada en cada iteración:

$$X(i) = (a * X(i-1) + c) \bmod m$$

$$RI(i) = X(i) / m$$

El método se invoca tres veces en `main()`: una para cada generador independiente (Llegadas, Inspección, Empaque).

3.3 Métodos siguienteRi*()

Tres métodos idénticos en estructura (siguienteRiLlegadas, siguienteRiInspeccion, siguienteRiEmpaque) que devuelven el próximo RI disponible de su array correspondiente. Cuando el índice supera el tamaño del array, se reinicia a 0 (comportamiento circular):

```
if (indiceLlegadas >= riLlegadas.length) indiceLlegadas = 0;
return riLlegadas[indiceLlegadas++];
```

Advertencia: El reinicio circular puede introducir correlación en secuencias largas si n es pequeño. Se recomienda que n sea suficientemente grande para cubrir todos los eventos esperados.

3.4 Métodos de Distribución

Implementan la transformada inversa para convertir un RI en $[0,1)$ a un tiempo de servicio con distribución deseada:

Método	Distribución	Parámetros	Formula	Uso
uniforme(min, max)	Uniforme continua	min=4, max=10	$\text{min} + (\text{max} - \text{min}) * \text{RI}$	Intervalo entre llegadas (minutos)
exponencialInspeccion(media)	Exponencial	media=5	$-\text{media} * \ln(1 - \text{RI})$	Tiempo de servicio en inspección (minutos)
exponencialEmpaque(media)	Exponencial	media=8	$-\text{media} * \ln(1 - \text{RI})$	Tiempo de servicio en empaque (minutos)

Nota técnica: El uso de $(1 - \text{RI})$ en la exponencial en lugar de RI directamente es equivalente estadísticamente, ya que si $\text{RI} \sim \text{Uniforme}(0,1)$, entonces $(1 - \text{RI})$ también lo es. Sin embargo hace que $\text{RI}=1$ provoque $\log(0)$. Esto no ocurre en la práctica con el LGC ya que $x=0$ genera $\text{RI}=0$, no $\text{RI}=1$.

4. Lógica de Eventos

4.1 Ciclo Principal de Simulación

El motor de simulación sigue el paradigma de avance por eventos (next-event time advance). En cada iteración del ciclo while se elige el evento con el menor tiempo futuro y se avanza el reloj a ese punto:

Versión Main (3 posibles eventos):

```
siguienteEvento = min(proximaLlegada, finInspeccion, finEmpaque)
```

Versión Prueba_2 (4 posibles eventos):

```
siguienteEvento = min(proximaLlegada, finInspeccion, finEmpaque1,  
finEmpaque2)
```

El ciclo termina cuando reloj $\geq$ TIEMPO_SIMULACION (2400 minutos). Los eventos cuyo tiempo es Double.MAX_VALUE (recurso inactivo) nunca se seleccionan como mínimo.

4.2 Evento: llegada()

Se dispara cuando reloj == proximaLlegada.

Paso	Accion
1	Incrementa piezasLlegadas.
2	Calcula la siguiente llegada: proximaLlegada = reloj + uniforme(4, 10).
3	Si inspector libre: lo ocupa, calcula tiempo de servicio exponencial(5), acumula tiempoOcupadoInspector, programa finInspeccion.
4	Si inspector ocupado: agrega 1 a colaInspeccion (cola FIFO ilimitada).

4.3 Evento: finInspeccion()

Se dispara cuando reloj == finInspeccion.

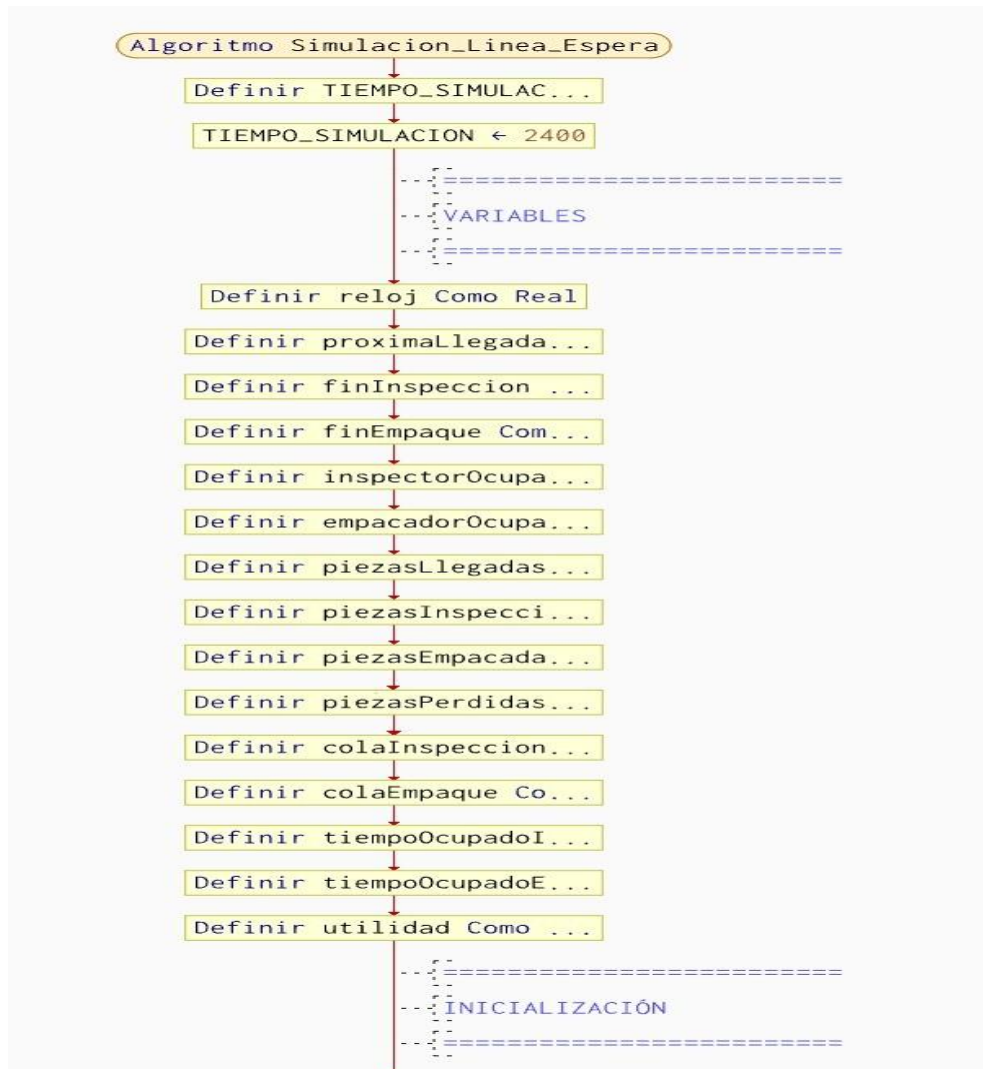
Paso	Accion
1	Incrementa piezasInspeccionadas.
2	Revisa colaEmpaque: si tiene menos de 5 piezas (limite solo en Prueba_2), intenta asignar a un empacador libre.
3 (Prueba_2)	Intenta Empacador1 primero; si ocupado, intenta Empacador2; si ambos ocupados, encola en colaEmpaque.
3 (Main)	Si el empacador unico esta libre: lo ocupa, calcula exponencial(8), acumula tiempo, programa finEmpaque. Si no: encola.
4	Si colaEmpaque.size() >= 5: la pieza se pierde (piezasPerdidas++). Solo aplica en Prueba_2; Main no tiene este limite explicito.
5	Revisa colaInspeccion: si no esta vacia, saca una pieza, calcula nuevo tiempo de inspeccion y programa finInspeccion. Si vacia, inspectorOcupado = false, finInspeccion = MAX_VALUE.

4.4 Evento: finEmpaque() / finEmpaque1() / finEmpaque2()

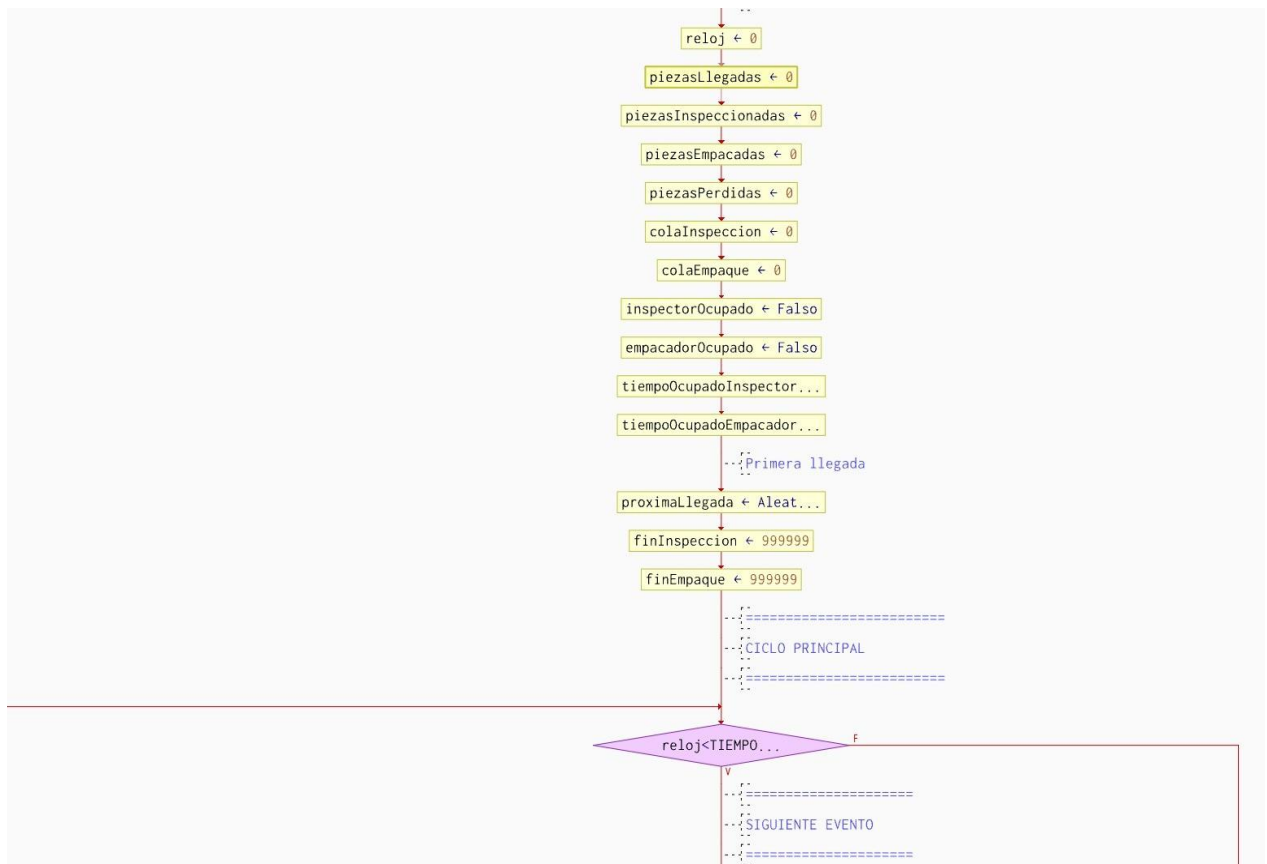
Se dispara cuando reloj == finEmpaque (Prueba_2) o finEmpaque1/finEmpaque2 (Main). La lógica es idéntica para los tres:

Paso	Accion
1	Incrementa piezasEmpacadas.
2	Revisa colaEmpaque: si no está vacía, saca una pieza, calcula nuevo tiempo de empaque exponencial(8), acumula tiempoOcupadoEmpacador, reprograma finEmpaque (o finEmpaque1/2).
3	Si colaEmpaque está vacía: empacadorOcupado = false (o empacador1/2Ocupado), finEmpaque = MAX_VALUE.

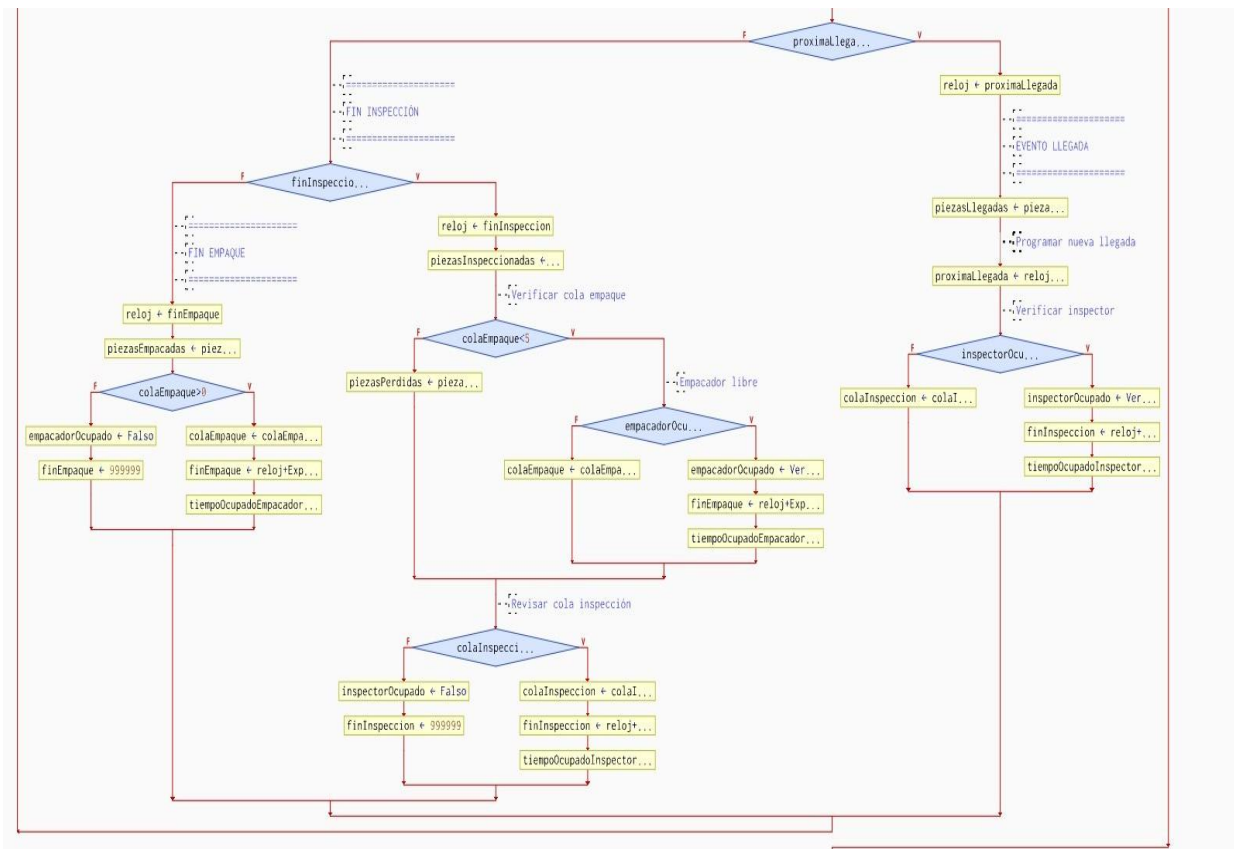
Diagrama de Flujo - Un Empaquetador



1. Iniciar algoritmo.
2. Definir variables.
3. Inicializar contadores y colas.
4. Inicializar reloj de simulación.
5. Definir recursos como libres.



1. Generar primera llegada.
2. Inicializar tiempos de eventos.
3. Iniciar ciclo principal.
4. Verificar tiempo de simulación.
5. Identificar siguiente evento.



Evento llegada

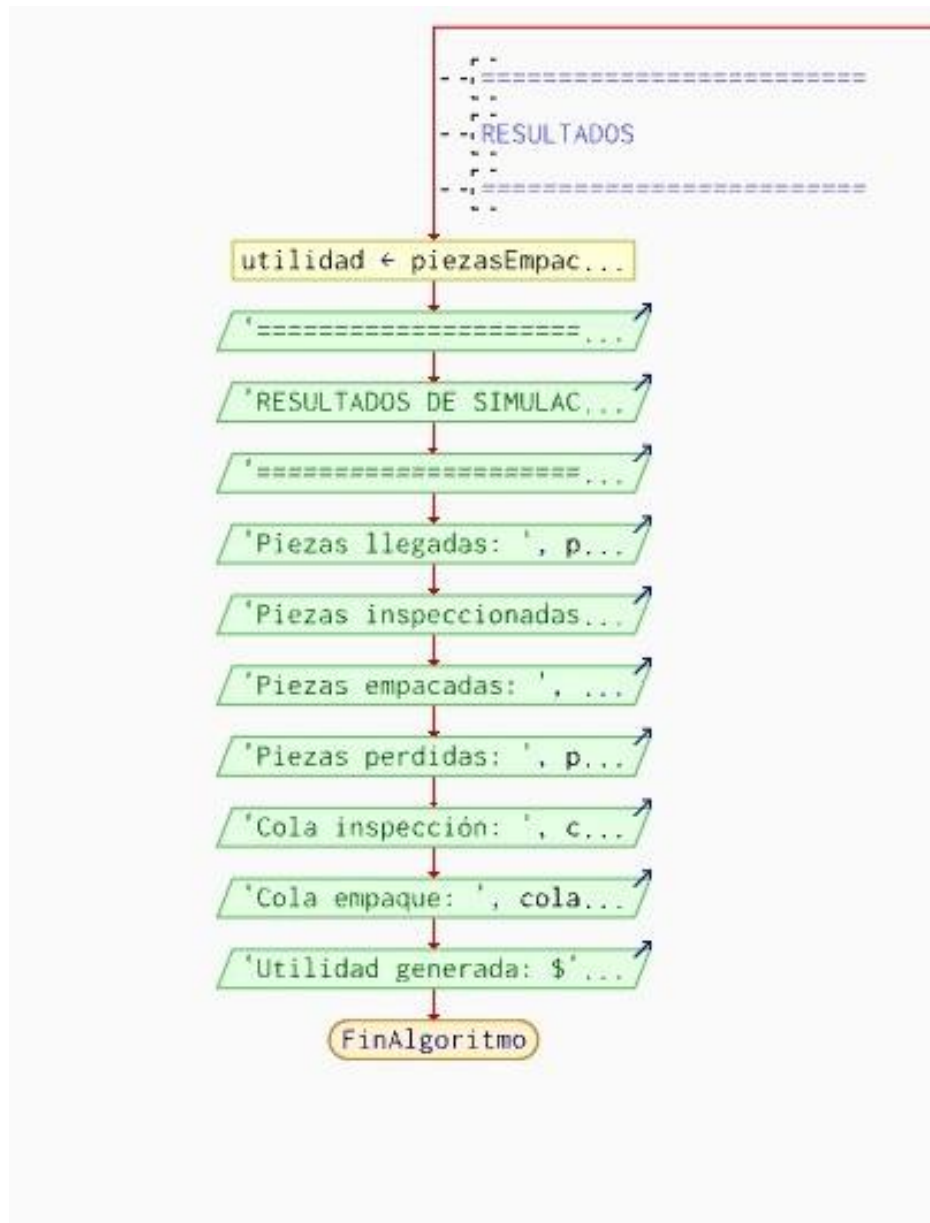
1. Registrar llegada de pieza.
2. Programar siguiente llegada.
3. Verificar inspector.
4. Enviar pieza a inspección o cola.

Evento fin inspección

1. Registrar inspección terminada.
2. Verificar espacio en empaque.
3. Enviar pieza a empaque o cola.
4. Revisar cola de inspección.

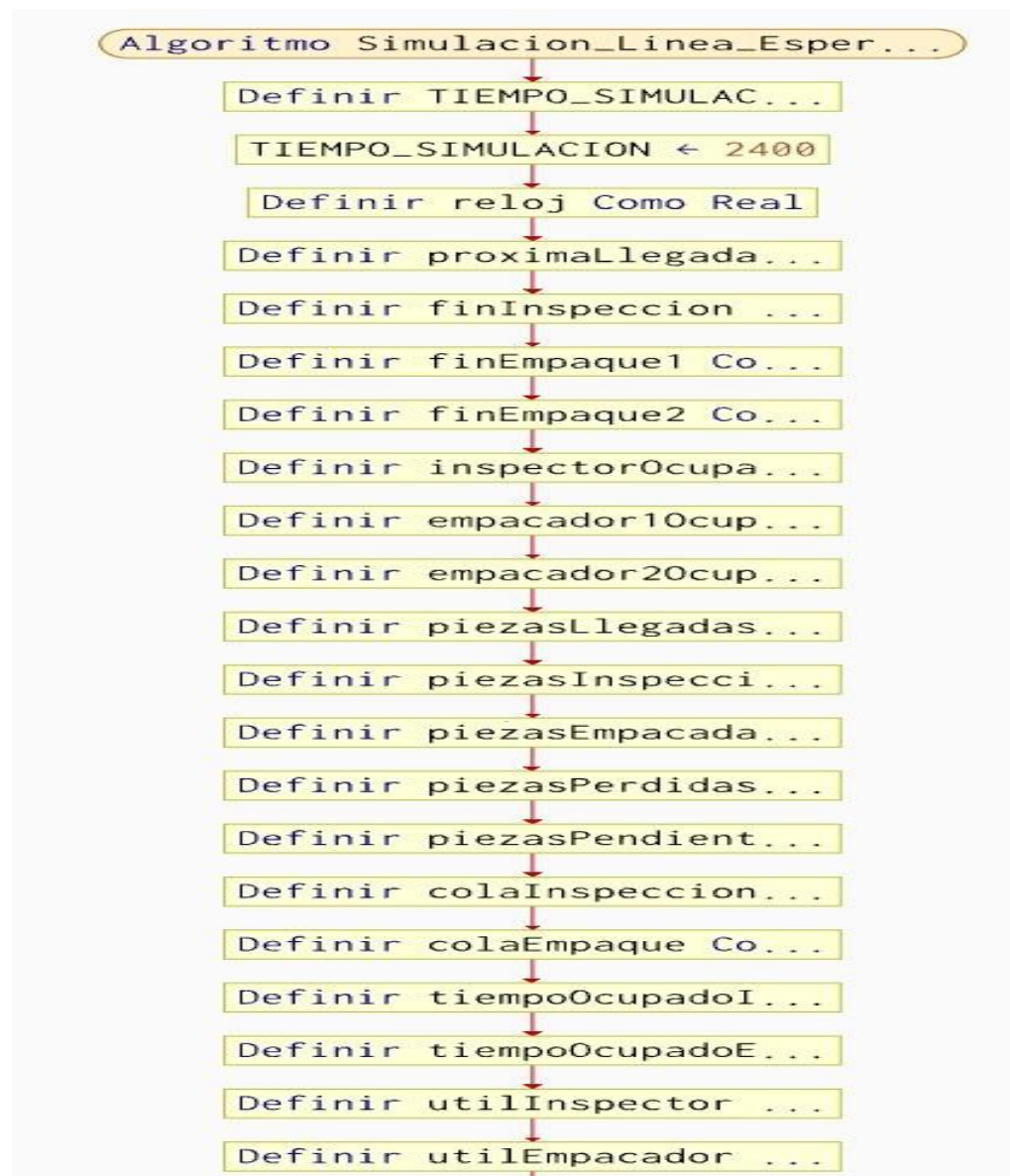
Evento fin empaque

1. Registrar pieza empacada.
2. Revisar cola de empaque.
3. Liberar empacador si no hay piezas.



1. Calcular utilidad.
2. Mostrar resultados de simulación.
3. Mostrar piezas procesadas y pérdidas.
4. Mostrar colas finales.
5. Finalizar algoritmo.

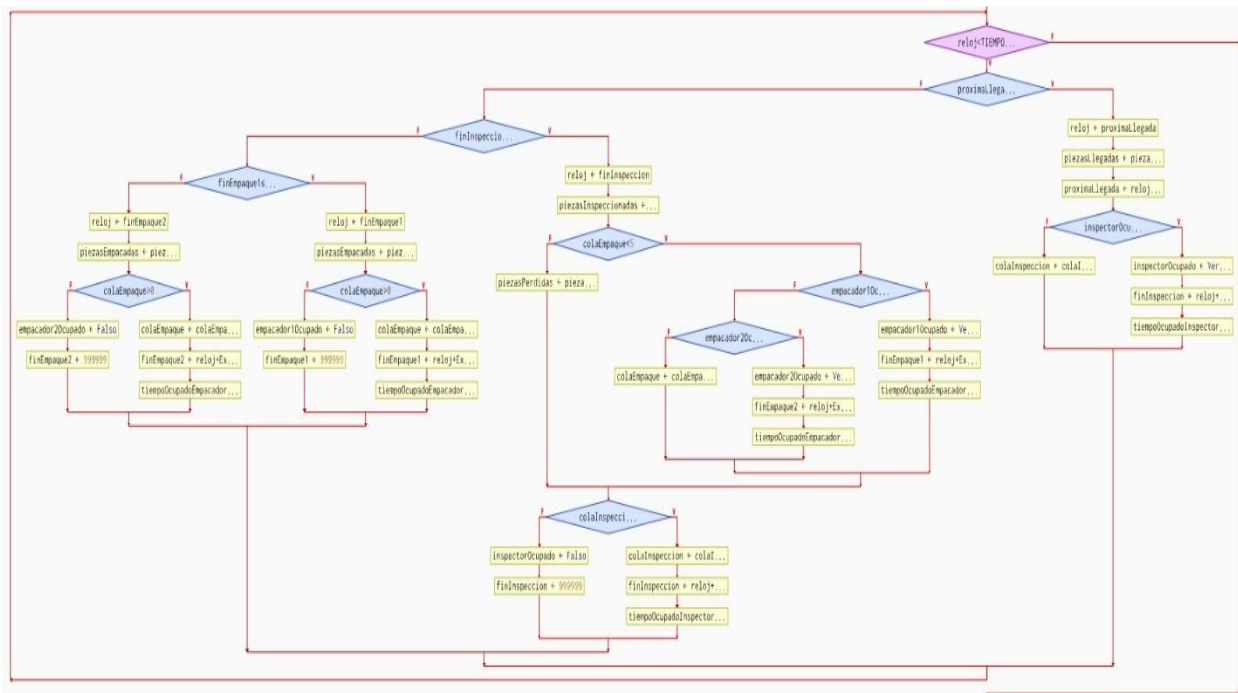
Diagrama de Flujo - Dos Empaquetadores



1. Iniciar algoritmo.
2. Definir tiempo de simulación.
3. Declarar variables principales.
4. Declarar eventos de inspección y empaque.
5. Declarar estados de inspector y empacadores.
6. Declarar contadores y colas.
7. Declarar variables de utilización y utilidad.



1. Inicializar reloj en 0.
2. Generar primera llegada.
3. Inicializar tiempos de eventos.
4. Colocar recursos como libres.
5. Inicializar contadores y colas en 0.
6. Iniciar ciclo principal.
7. Verificar siguiente evento.



Evento llegada

1. Registrar llegada de pieza.
2. Programar nueva llegada.
3. Verificar inspector.
4. Enviar pieza a inspección o cola.

Evento fin inspección

1. Registrar inspección terminada.
2. Verificar cola de empaque.
3. Verificar empacador 1.
4. Verificar empacador 2.
5. Enviar pieza a empaque o cola.
6. Revisar cola de inspección.

Evento fin empaque

1. Registrar pieza empacada.
2. Revisar cola de empaque.
3. Liberar empacador si no hay piezas



1. Calcular piezas pendientes.
2. Calcular utilización del inspector.
3. Calcular utilización de empacadores.
4. Calcular utilidad generada.
5. Mostrar resultados finales.
6. Finalizar algoritmo.

5. Códigos Fuentes

5.1 Main

```
package Main;
/**Autoría:   Gutiérrez Rivas Alejandro
 *           Herrera Martinez Aylin
 *           Vázquez Alviso Bryan Alexei */
/*
```

----- ENUNCIADO DEL PROBLEMA

10. A un sistema llegan piezas de acuerdo con una distribución uniforme de entre 4 y 10 minutos.

Las piezas son colocadas en un almacén con capacidad infinita, donde esperan a ser inspeccionadas por un operario.

El tiempo de inspección tiene una distribución exponencial con media de 5 minutos.

Después de la inspección las piezas pasan a la fila de empaque, con capacidad para 5 piezas.

El proceso de empaque está a cargo de un operario que tarda 8 minutos con distribución exponencial

en empaquetar cada pieza. Posteriormente las piezas salen del sistema.

a) Simule el sistema por 40 horas.

b) Identifique dónde se encuentra el cuello de botella.

c) Genere vistas para cada uno de los procesos por separado.

d) Incrementar el espacio en el almacén cuesta \$5/semana;
aumentar 10% la velocidad de empaque cuesta \$15/semana;
el costo de incluir otro operario para que se reduzca el tiempo de empaque a 5 minutos con distribución exponencial, es de \$20/semana.

Cada unidad producida deja una utilidad de \$0.40.

Con base en esta información, determine qué mejoras podrían hacerse al sistema para incrementar su utilidad semanal.

```
-----
*/
import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;
public class Main {
    // =====
    // TIEMPO TOTAL DE SIMULACIÓN
    // =====
    static final double TIEMPO_SIMULACION = 2400.0;

    // =====
    // GENERADORES INDEPENDIENTES
    // =====
    // Para cada proceso se tiene un generador independiente de números aleatorios
    static double[] riLlegadas;
    static double[] riInspeccion;
    static double[] riEmpaque;
    // Índices para cada generador
```

```
static int indiceLlegadas = 0;
static int indiceInspeccion = 0;
static int indiceEmpaque = 0;

// =====
// RELOJ DE SIMULACIÓN
// =====
// El reloj de simulación se actualiza al ocurrir cada evento
static double reloj = 0;

// =====
// EVENTOS FUTUROS
// =====
// Se mantiene el tiempo del próximo evento de cada tipo
static double proximaLlegada;
static double finInspeccion = Double.MAX_VALUE;
static double finEmpaque = Double.MAX_VALUE;

// =====
// ESTADO DE RECURSOS
// =====
// Se mantiene el estado de cada recurso (ocupado o libre)
static boolean inspectorOcupado = false;
static boolean empacadorOcupado = false;

// =====
// COLAS
// =====
// Se mantienen colas para cada proceso, aunque en este caso solo el empaque tiene una
// capacidad limitada
static Queue<Integer> colaInspeccion =
    new LinkedList<>();

static Queue<Integer> colaEmpaque =
    new LinkedList<>();

// =====
// CONTADORES
// =====
// Se mantienen contadores para las piezas que llegan, son inspeccionadas, empacadas y
// perdidas
static int piezasLlegadas = 0;
static int piezasInspeccionadas = 0;
static int piezasEmpacadas = 0;
static int piezasPerdidas = 0;

// =====
// ACUMULADORES
// =====
// Se mantienen acumuladores para el tiempo ocupado de cada recurso, lo que permitirá
// calcular su utilización
```

```
static double tiempoOcupadoInspector = 0;
static double tiempoOcupadoEmpacador = 0;

// =====
// GENERADOR CONGRUENCIAL LINEAL
// =====
// Este método genera un arreglo de números pseudoaleatorios utilizando el método
congruencial lineal
public static double[] generadorLGC() {

    Scanner sc = new Scanner(System.in);

    long a, c, x, m;
    int n;

    System.out.print("Ingresa a: ");
    a = sc.nextLong();

    System.out.print("Ingresa c: ");
    c = sc.nextLong();

    System.out.print("Ingresa x0: ");
    x = sc.nextLong();

    System.out.print("Ingresa m: ");
    m = sc.nextLong();

    System.out.print("Cantidad de números: ");
    n = sc.nextInt();
// Se genera el arreglo de números pseudoaleatorios uniformes
    double[] ri = new double[n];

    for (int i = 0; i < n; i++) {

        x = (a * x + c) % m;

        ri[i] = (double) x / m;
    }
    return ri;
}

// =====
// RI PARA LLEGADAS
// =====
/* Este método devuelve el siguiente número pseudoaleatorio para las llegadas,
y reinicia el índice si se alcanza el final del arreglo
*/
public static double siguienteRiLlegadas() {
// Si se alcanza el final del arreglo, se reinicia el índice para reutilizar los números
    if (indiceLlegadas >= riLlegadas.length) {
```

```
        indiceLlegadas = 0;
    }
// Se devuelve el siguiente número pseudoaleatorio para las llegadas
    return riLlegadas[indiceLlegadas++];
}

// =====
// RI PARA INSPECCIÓN
// =====
// Este método devuelve el siguiente número pseudoaleatorio para la inspección,
public static double siguienteRiInspeccion() {
// y reinicia el índice si se alcanza el final del arreglo
    if (indiceInspeccion >= riInspeccion.length) {
        indiceInspeccion = 0;
    }
    return riInspeccion[indiceInspeccion++];
}

// =====
// RI PARA EMPAQUE
// =====
// Este método devuelve el siguiente número pseudoaleatorio para el empaque,
public static double siguienteRiEmpaque() {
// y reinicia el índice si se alcanza el final del arreglo
    if (indiceEmpaque >= riEmpaque.length) {
        indiceEmpaque = 0;
    }
    return riEmpaque[indiceEmpaque++];
}

// =====
// DISTRIBUCIÓN UNIFORME
// =====
/* Este método genera un número aleatorio con distribución uniforme entre min y max
    utilizando el siguiente número pseudoaleatorio para las llegadas
*/
public static double uniforme(double min,
                             double max) {

    double r = siguienteRiLlegadas();
// Se transforma el número pseudoaleatorio uniforme en un número con distribución uniforme
entre min y max
    return min + (max - min) * r;
}

// =====
// DISTRIBUCIÓN EXPONENCIAL INSPECCIÓN
// =====
/* Este método genera un número aleatorio con distribución exponencial
    utilizando el siguiente número pseudoaleatorio para la inspección*/
public static double exponencialInspeccion(
    double media) {
```

```
        double r = siguienteRiInspeccion();
/* Se transforma el número pseudoaleatorio uniforme
   en un número con distribución exponencial con la media especificada
*/
        return -media * Math.log(1 - r);
    }

    // =====
    // DISTRIBUCIÓN EXPONENCIAL EMPAQUE
    // =====
    // Este método genera un número aleatorio con distribución exponencial
    public static double exponencialEmpaque(
        double media) {
    // utilizando el siguiente número pseudoaleatorio para el empaque
        double r = siguienteRiEmpaque();
    // Se transforma el número pseudoaleatorio uniforme
        return -media * Math.log(1 - r);
    }

    // =====
    // EVENTO LLEGADA
    // =====
    // Este método se ejecuta cada vez que ocurre una llegada de pieza al sistema
    public static void llegada() {
    // Al llegar una pieza, se incrementa el contador de piezas llegadas
        piezasLlegadas++;
    // Se programa la próxima llegada generando un nuevo tiempo de llegada
        proximaLlegada =
            reloj + uniforme(4, 10);
    // Al llegar una pieza, se verifica si el inspector está ocupado
    if (!inspectorOcupado) {
    // Si el inspector no está ocupado, la pieza se inspecciona inmediatamente
        inspectorOcupado = true;
    // Se genera el tiempo de servicio para la inspección
        double tiempoServicio =
            exponencialInspeccion(5);
    // Se acumula el tiempo ocupado del inspector
        tiempoOcupadoInspector +=
            tiempoServicio;

        finInspeccion =
            reloj + tiempoServicio;
    } else {
    // Si el inspector está ocupado, la pieza se agrega a la cola de inspección
        colaInspeccion.add(1);
    }
    }

    // =====
    // EVENTO FIN INSPECCIÓN
    // =====
```

```
public static void finInspeccion() {
// Al finalizar la inspección, se incrementa el contador de piezas inspeccionadas
piezasInspeccionadas++;
// Después de la inspección, la pieza pasa a la fila de empaque
if (colaEmpaque.size() < 5) {
// Si la cola de empaque tiene espacio, se verifica si el empacador está ocupado
if (!empacadorOcupado) {
// Si el empacador no está ocupado, la pieza se empaca inmediatamente
empacadorOcupado = true;
// Se genera el tiempo de servicio para el empaque
double tiempoEmpaque =
    exponencialEmpaque(8);
// Se acumula el tiempo ocupado del empacador
tiempoOcupadoEmpacador +=
    tiempoEmpaque;
// Se programa el fin del empaque para esta pieza
finEmpaque =
    reloj + tiempoEmpaque;

} else {
// Si el empacador está ocupado, la pieza se agrega a la cola de empaque
colaEmpaque.add(1);
}
// Si la cola de empaque está llena, la pieza se pierde
} else {
    piezasPerdidas++;
}
// Después de procesar la pieza actual, se verifica si hay piezas esperando en la cola de
inspección
if (!colaInspeccion.isEmpty()) {
    colaInspeccion.poll();

    double tiempoServicio =
        exponencialInspeccion(5);

    tiempoOcupadoInspector +=
        tiempoServicio;

    finInspeccion =
        reloj + tiempoServicio;

} else {

    inspectorOcupado = false;

    finInspeccion =
        Double.MAX_VALUE;
}
}
```

```
// =====
// EVENTO FIN EMPAQUE
// =====
// Al finalizar el empaque, se incrementa el contador de piezas empacadas
public static void finEmpaque() {

    piezasEmpacadas++;
// Después de procesar la pieza actual, se verifica si hay piezas esperando en la cola de
empaque
    if (!colaEmpaque.isEmpty()) {

        colaEmpaque.poll();
// Se genera el tiempo de servicio para el empaque de la siguiente pieza
        double tiempoEmpaque =
            exponencialEmpaque(8);
// Se acumula el tiempo ocupado del empacador
        tiempoOcupadoEmpacador +=
            tiempoEmpaque;
// Se programa el fin del empaque para la siguiente pieza
        finEmpaque =
            reloj + tiempoEmpaque;

    } else {
// Si no hay piezas esperando, el empacador queda libre
        empacadorOcupado = false;
// Se programa el fin del empaque para un tiempo infinito, ya que no hay piezas por empacar
        finEmpaque =
            Double.MAX_VALUE;
    }
}

// =====
// MAIN
// =====
/*Este es el método principal donde se inicializan los generadores,
se ejecuta el ciclo de simulación y se muestran los resultados*/
public static void main(String[] args) {

    Scanner pausa = new Scanner(System.in);

    System.out.println(
        "GENERADOR LLEGADAS");

    riLlegadas = generadorLGC();

    System.out.println(
        "\nGENERADOR INSPECCIÓN");

    riInspeccion = generadorLGC();

    System.out.println(
```

```
"\nGENERADOR EMPAQUE");

riEmpaque = generadorLGC();

// Primera llegada al sistema
proximaLlegada = uniforme(4, 10);

// =====
// CICLO PRINCIPAL
// =====
// El ciclo principal de la simulación se ejecuta hasta que el reloj alcance el tiempo total de
simulación
while (reloj < TIEMPO_SIMULACION) {
// Se determina cuál es el siguiente evento a ocurrir comparando los tiempos de los próximos
eventos
    double siguienteEvento = Math.min(
        proximaLlegada,
        Math.min(
            finInspeccion,
            finEmpaque
        )
    );
// Se avanza el reloj al tiempo del siguiente evento
    reloj = siguienteEvento;

    if (reloj == proximaLlegada) {

        llegada();

    } else if (reloj == finInspeccion) {

        finInspeccion();

    } else if (reloj == finEmpaque) {

        finEmpaque();

    }
}

// =====
// RESULTADOS
// =====

System.out.println(
    "\n=====");

System.out.println(
    "RESULTADOS DE SIMULACIÓN");

System.out.println(
    "=====");
```



```
System.out.println(
    "Piezas llegadas: "
        + piezasLlegadas);

System.out.println(
    "Piezas inspeccionadas: "
        + piezasInspeccionadas);

System.out.println(
    "Piezas empacadas: "
        + piezasEmpacadas);

System.out.println(
    "Piezas perdidas: "
        + piezasPerdidas);

// =====
// PIEZAS PENDIENTES
// =====
// Se calcula el número de piezas pendientes en el sistema al finalizar la simulación
int piezasPendientes =
    piezasLlegadas
        - piezasEmpacadas
        - piezasPerdidas;
// Se muestra el número de piezas pendientes en el sistema
System.out.println(
    "Piezas pendientes: "
        + piezasPendientes);

System.out.println();
// Se muestra el número de piezas en cada cola al finalizar la simulación
System.out.println(
    "Cola final inspección: "
        + colaInspeccion.size());

System.out.println(
    "Cola final empaque: "
        + colaEmpaque.size());

// =====
// UTILIZACIÓN
// =====
// Se calcula la utilización de cada recurso dividiendo el tiempo ocupado por el tiempo total de
simulación
double utilisInspector =
    tiempoOcupadoInspector
        / TIEMPO_SIMULACION;

double utilisEmpacador =
    tiempoOcupadoEmpacador
```

```
        / TIEMPO_SIMULACION;

    System.out.printf(
        "\nUtilización inspector: %.2f%%\n",
        utilInspector * 100
    );

    System.out.printf(
        "Utilización empacador: %.2f%%\n",
        utilEmpacador * 100
    );

    // =====
    // UTILIDAD
    // =====
    /*Se calcula la utilidad generada por las piezas empacadas
    multiplicando el número de piezas empacadas por la utilidad por pieza*/
    double utilidad =
        piezasEmpacadas * 0.40;

    System.out.printf(
        "\nUtilidad generada: $%.2f\n",
        utilidad
    );

    // =====
    // CUELLO DE BOTELLA
    // =====
    // Se identifica el cuello de botella comparando las utilizaciones de los recursos
    if (utilEmpacador >
        utilInspector) {

        System.out.println(
            "\nCuello de botella: EMPAQUE");

    } else {

        System.out.println(
            "\nCuello de botella: INSPECCIÓN");
    }

    // =====
    // EVITAR QUE SE CIERRE
    // =====
    /*Se muestra un mensaje para evitar que la consola se cierre inmediatamente
    después de mostrar los resultados*/
    System.out.println(
        "\nPresiona ENTER para salir...");
    pausa.nextLine();
}
}
```

5.2 Prueba_2

```
package Prueba_2;
/**Autoría:   Gutiérrez Rivas Alejandro
 *           Herrera Martinez Aylin
 *           Vázquez Alviso Bryan Alexei */
/*
```

----- ENUNCIADO DEL PROBLEMA

10. A un sistema llegan piezas de acuerdo con una distribución uniforme de entre 4 y 10 minutos.

Las piezas son colocadas en un almacén con capacidad infinita, donde esperan a ser inspeccionadas por un operario.

El tiempo de inspección tiene una distribución exponencial con media de 5 minutos.

Después de la inspección las piezas pasan a la fila de empaque, con capacidad para 5 piezas.

El proceso de empaque está a cargo de un operario que tarda 8 minutos con distribución exponencial

en empaacar cada pieza. Posteriormente las piezas salen del sistema.

a) Simule el sistema por 40 horas.

b) Identifique dónde se encuentra el cuello de botella.

c) Genere vistas para cada uno de los procesos por separado.

d) Incrementar el espacio en el almacén cuesta \$5/semana;
aumentar 10% la velocidad de empaque cuesta \$15/semana;
el costo de incluir otro operario para que se reduzca el tiempo de empaque a 5 minutos con distribución exponencial, es de \$20/semana.

Cada unidad producida deja una utilidad de \$0.40.

Con base en esta información, determine qué mejoras podrían hacerse al sistema para incrementar su utilidad semanal.

```
-----
*/
import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;

public class Prueba_2 {
    // =====
    // VARIABLES GLOBALES
    // =====
    static final double TIEMPO_SIMULACION = 2400.0;
    static double[] riLlegadas;
    static double[] riInspeccion;
    static double[] riEmpaque;
    static int indiceLlegadas = 0;
    static int indiceInspeccion = 0;
    static int indiceEmpaque = 0;
    static double reloj = 0;
    static double proximaLlegada;
    static double finInspeccion = Double.MAX_VALUE;
    static double finEmpaque1 = Double.MAX_VALUE;
    static double finEmpaque2 = Double.MAX_VALUE;
```

```
static boolean inspectorOcupado = false;
static boolean empacador1Ocupado = false;
static boolean empacador2Ocupado = false;
static Queue<Integer> colaInspeccion = new LinkedList<>();
static Queue<Integer> colaEmpaque = new LinkedList<>();
static int piezasLlegadas = 0;
static int piezasInspeccionadas = 0;
static int piezasEmpacadas = 0;
static int piezasPerdidas = 0;
static double tiempoOcupadoInspector = 0;
static double tiempoOcupadoEmpacador = 0;
// =====
// MÉTODOS
// =====
// Método para generar números pseudoaleatorios utilizando el método LGC
public static double[] generadorLGC() {
    Scanner sc = new Scanner(System.in);
    long a, c, x, m;
    int n;

    System.out.print("Ingresa a: ");
    a = sc.nextLong();

    System.out.print("Ingresa c: ");
    c = sc.nextLong();

    System.out.print("Ingresa x0: ");
    x = sc.nextLong();

    System.out.print("Ingresa m: ");
    m = sc.nextLong();

    System.out.print("Cantidad de números: ");
    n = sc.nextInt();
    // Se crea un arreglo para almacenar los números pseudoaleatorios uniformes generados
    double[] ri = new double[n];

    for (int i = 0; i < n; i++) {
        x = (a * x + c) % m;
        ri[i] = (double) x / m;
    }

    return ri;
}
// Método para imprimir los resultados del generador de números pseudoaleatorios
public static void imprimirGenerador(String nombre, double[] ri, String tipo) {
    System.out.println("\n=====");
    System.out.println(nombre);
    System.out.println("=====");
}
/*Se identifica el tipo de evento para calcular los valores correspondientes
utilizando las fórmulas de distribución uniforme o exponencial
```

```
*/  
// Las llegadas se generan con distribución uniforme entre 4 y 10 minutos  
if (tipo.equals("LLEGADAS")) {  
    System.out.println("No.\tRi\t\tUniforme(4,10)");  
    //Se imprimen los números pseudoaleatorios y sus transformaciones correspondientes  
    for (int i = 0; i < ri.length; i++) {  
        double valor = 4 + (10 - 4) * ri[i];  
        System.out.printf("%d\t%.5f\t%.5f\n",  
            i + 1,  
            ri[i],  
            valor);  
    }  
    // Las inspecciones se generan con distribución exponencial con media 5 minutos  
    } else if (tipo.equals("INSPECCION")) {  
        System.out.println("No.\tRi\t\tExponencial(5)");  
        // Se imprimen los números pseudoaleatorios y sus transformaciones correspondientes  
        for (int i = 0; i < ri.length; i++) {  
            double valor = -5 * Math.log(1 - ri[i]);  
            System.out.printf("%d\t%.5f\t%.5f\n",  
                i + 1,  
                ri[i],  
                valor);  
        }  
        // El empaque se genera con distribución exponencial con media 8 minutos  
        } else if (tipo.equals("EMPAQUE")) {  
            System.out.println("No.\tRi\t\tExponencial(8)");  
            // Se imprimen los números pseudoaleatorios y sus transformaciones correspondientes  
            for (int i = 0; i < ri.length; i++) {  
                double valor = -8 * Math.log(1 - ri[i]);  
                System.out.printf("%d\t%.5f\t%.5f\n",  
                    i + 1,  
                    ri[i],  
                    valor);  
            }  
        }  
    }  
}  
// Métodos para obtener el siguiente número pseudoaleatorio de cada generador,  
// reiniciando el índice si se alcanza el final del arreglo  
public static double siguienteRiLlegadas() {  
    if (indiceLlegadas >= riLlegadas.length) {  
        indiceLlegadas = 0;  
    }  
    return riLlegadas[indiceLlegadas++];  
}  
public static double siguienteRiInspeccion() {  
    if (indiceInspeccion >= riInspeccion.length) {  
        indiceInspeccion = 0;  
    }  
    return riInspeccion[indiceInspeccion++];  
}  
public static double siguienteRiEmpaque() {
```

```
        if (indiceEmpaque >= riEmpaque.length) {
            indiceEmpaque = 0;
        }
        return riEmpaque[indiceEmpaque++];
    }
// Métodos para generar los tiempos de llegada,
// inspección y empaque utilizando las fórmulas de distribución correspondientes
// La llegada se genera con distribución uniforme entre 4 y 10 minutos
public static double uniforme(double min, double max) {
    double r = siguienteRiLlegadas();
    return min + (max - min) * r;
}
// La inspección se genera con distribución exponencial con media 5 minutos
public static double exponencialInspeccion(double media) {
    double r = siguienteRiInspeccion();
    return -media * Math.log(1 - r);
}
// El empaque se genera con distribución exponencial con media 8 minutos
public static double exponencialEmpaque(double media) {
    double r = siguienteRiEmpaque();
    return -media * Math.log(1 - r);
}
// Métodos para manejar los eventos de llegada, fin de inspección y fin de empaque
public static void llegada() {
    piezasLlegadas++;
    proximaLlegada = reloj + uniforme(4, 10);
    // Si el inspector no está ocupado, se inicia la inspección de la pieza
    if (!inspectorOcupado) {
        // Se marca el inspector como ocupado
        inspectorOcupado = true;
        // Se genera el tiempo de inspección para la pieza y se acumula el tiempo ocupado por el
        // inspector
        double t = exponencialInspeccion(5);
        tiempoOcupadoInspector += t;
        // Se programa el fin de inspección para la pieza
        finInspeccion = reloj + t;
    } // Si el inspector está ocupado, la pieza se agrega a la cola de inspección
    else {
        colaInspeccion.add(1);
    }
}
// Método para manejar el evento de fin de inspección
public static void finInspeccion() {
    // Se incrementa el contador de piezas inspeccionadas
    piezasInspeccionadas++;
    // Si hay piezas en la cola de inspección, se inicia la inspección de la siguiente pieza
    if (colaEmpaque.size() < 5) {
        // Si el empacador 1 no está ocupado, se asigna la pieza al empacador 1
        if (!empacador1Ocupado) {
            // Se marca el empacador 1 como ocupado
            empacador1Ocupado = true;
```

```
// Se genera el tiempo de empaque para la pieza y se acumula el tiempo ocupado por el
empacador
    double t = exponencialEmpaque(8);
// Se acumula el tiempo ocupado por el empacador para calcular la utilización
    tiempoOcupadoEmpacador += t;
// Se programa el fin de empaque para el empacador 1
    finEmpaque1 = reloj + t;

}
// Si el empacador 1 está ocupado pero el empacador 2 no está ocupado,
// se asigna la pieza al empacador 2
    else if (!empacador2Ocupado) {
        empacador2Ocupado = true;
        double t = exponencialEmpaque(8);
        tiempoOcupadoEmpacador += t;
        finEmpaque2 = reloj + t;
    }
// Si ambos empacadores están ocupados,
// la pieza se agrega a la cola de empaque
    else {
        colaEmpaque.add(1);
    }
// Si la cola de empaque tiene 5 piezas, la pieza se pierde
    } else {
        piezasPerdidas++;
    }
}
// Si hay piezas en la cola de inspección, se inicia la inspección de la siguiente pieza
    if (!colaInspeccion.isEmpty()) {
        colaInspeccion.poll();
        double t = exponencialInspeccion(5);
        tiempoOcupadoInspector += t;
        finInspeccion = reloj + t;
    } else {
        // Si no hay piezas en la cola de inspección, el inspector queda libre
        inspectorOcupado = false;
        // Se programa el fin de inspección con un valor muy alto para indicar que no hay evento
        programado
        finInspeccion = Double.MAX_VALUE;
    }
}
// Métodos para manejar los eventos de fin de empaque para cada empacador
public static void finEmpaque1() {
    // Se incrementa el contador de piezas empacadas
    piezasEmpacadas++;
    // Si hay piezas en la cola de empaque, se inicia el empaque de la siguiente pieza
    if (!colaEmpaque.isEmpty()) {
        colaEmpaque.poll();
        double t = exponencialEmpaque(8);
        tiempoOcupadoEmpacador += t;
        finEmpaque1 = reloj + t;
    } else {
```

```
// Si no hay piezas en la cola de empaque, el empacador 1 queda libre
    empacador1Ocupado = false;
// Se programa el fin de empaque con un valor muy alto para indicar que no hay evento
programado
    finEmpaque1 = Double.MAX_VALUE;
}
}
// Método para manejar el evento de fin de empaque para el empacador 2
public static void finEmpaque2() {
    piezasEmpacadas++;
    if (!colaEmpaque.isEmpty()) {
        colaEmpaque.poll();
        double t = exponencialEmpaque(8);
        tiempoOcupadoEmpacador += t;
        finEmpaque2 = reloj + t;
    } else {
        empacador2Ocupado = false;
        finEmpaque2 = Double.MAX_VALUE;
    }
}
// =====
// MAIN
// =====
public static void main(String[] args) {
    // Se crea un objeto Scanner para pausar la consola al finalizar la simulación
    Scanner pausa = new Scanner(System.in);
    // Se generan los números pseudoaleatorios para las llegadas,
    // inspección y empaque utilizando el método LGC
    System.out.println("GENERADOR LLEGADAS");
    riLlegadas = generadorLGC();
    System.out.println("\nGENERADOR INSPECCIÓN");
    riInspeccion = generadorLGC();
    System.out.println("\nGENERADOR EMPAQUE");
    riEmpaque = generadorLGC();
    // Se imprimen los resultados de los generadores de números pseudoaleatorios
    imprimirGenerador(
        "RESULTADOS GENERADOR LLEGADAS",
        riLlegadas,
        "LLEGADAS"
    );
    imprimirGenerador(
        "RESULTADOS GENERADOR INSPECCIÓN",
        riInspeccion,
        "INSPECCION"
    );
    imprimirGenerador(
        "RESULTADOS GENERADOR EMPAQUE",
        riEmpaque,
        "EMPAQUE"
    );
    // Se inicializa el reloj y se programa la primera llegada
```



```
    proximaLlegada = uniforme(4, 10);
// Se ejecuta el ciclo de eventos hasta que se alcance el tiempo de simulación
    while (reloj < TIEMPO_SIMULACION) {
        // Se determina el siguiente evento a procesar comparando los tiempos de llegada,
        double siguienteEvento = Math.min(
            proximaLlegada,
            Math.min(
                finInspeccion,
                Math.min(finEmpaque1, finEmpaque2)
            )
        );
        // Se avanza el reloj al tiempo del siguiente evento
        reloj = siguienteEvento;
        // Se procesa el evento correspondiente según el tipo de evento identificado
        if (reloj == proximaLlegada) {
            llegada();

        } else if (reloj == finInspeccion) {
            finInspeccion();

        } else if (reloj == finEmpaque1) {
            finEmpaque1();

        } else if (reloj == finEmpaque2) {
            finEmpaque2();
        }
    }
// Se muestran los resultados de la simulación, incluyendo el número de piezas llegadas,
    System.out.println("\n=====");
    System.out.println("RESULTADOS DE SIMULACIÓN");
    System.out.println("=====");
    System.out.println("Piezas llegadas: " + piezasLlegadas);
    System.out.println("Piezas inspeccionadas: " + piezasInspeccionadas);
    System.out.println("Piezas empacadas: " + piezasEmpacadas);
    System.out.println("Piezas perdidas: " + piezasPerdidas);
    int piezasPendientes =
        piezasLlegadas - piezasEmpacadas - piezasPerdidas;
    System.out.println("Piezas pendientes: " + piezasPendientes);
    System.out.println("\nCola inspección: " + colaInspeccion.size());
    System.out.println("Cola empaque: " + colaEmpaque.size());
// Se calcula la utilización de cada recurso dividiendo el tiempo ocupado por el tiempo total de
simulación
    double utilInspector =
        tiempoOcupadoInspector / TIEMPO_SIMULACION;
    double utilEmpacador =
        tiempoOcupadoEmpacador / (TIEMPO_SIMULACION * 2);
// Se muestra la utilización de cada recurso en porcentaje
    System.out.printf(
        "\nUtilización inspector: %.2f%%\n",
        utilInspector * 100
    );
};
```

```
        System.out.printf(
            "Utilización empacadores: %.2f%%\n",
            utilEmpacador * 100
        );
// Se calcula la utilidad generada por el proceso
// asumiendo que cada pieza empacada genera una utilidad de $0.40
double utilidad = piezasEmpacadas * 0.40;

        System.out.printf(
            "\nUtilidad generada: $%.2f\n",
            utilidad
        );
// =====
// CUELLO DE BOTELLA
// =====
// Se identifica el cuello de botella comparando las utilizaciones de los recursos
if (utilEmpacador >
    utilInspector) {

    System.out.println(
        "\nCuello de botella: EMPAQUE");

} else {

    System.out.println(
        "\nCuello de botella: INSPECCIÓN");
}
System.out.println("\nPresiona ENTER para salir...");
pausa.nextLine();
}
}
```

6. Diferencias entre Main y Prueba_2

Aspecto	Main	Prueba_2
Empacadores	1 empacador único	2 empacadores independientes (Empacador1 y Empacador2)
Eventos de empaque	finEmpaque	finEmpaque1, finEmpaque2
Limite cola empaque	Máximo 5 piezas; si se supera, la pieza se pierde	Máximo 5 piezas; si se supera, la pieza se pierde
Piezas perdidas	Se cuentan en piezasPerdidas cuando colaEmpaque.size() >= 5	No aplica (nunca se descarta por cola llena)
Salida de generadores	No imprime tabla de generadores; va directo a la simulación	Imprime tabla con RI y valores de distribución por cada generador
Reportes de utilización	Si (inspector y empacador único)	Si (inspector y empacadores)
Cuello de botella	Calculado: si utilEmpacador > utilInspector es EMPAQUE, si no INSPECCION	Calculado: si utilEmpacador > utilInspector es EMPAQUE, si no INSPECCION
Utilidad económica	\$0.40 por pieza empacada (idéntico)	\$0.40 por pieza empacada
Pausa al salir	Si (ENTER para salir)	Si (ENTER para salir)
Clase Java	Main.Main	Prueba_2.Prueba_2

7. Salida del Programa

7.1 Salida de Main

La version Main no imprime tablas de generadores. Solo muestra el resumen ejecutivo al finalizar:

```
=====
RESULTADOS DE SIMULACION
=====
Piezas llegadas: NNN
Piezas inspeccionadas: NNN
Piezas empacadas: NNN
Piezas perdidas: NNN
Piezas pendientes: NNN

Cola final inspección: N
Cola final empaque: N

Utilización inspector: XX.XX%
Utilización empacador: XX.XX%

Utilidad generada: $XXX.XX

Cuello de botella: EMPAQUE    <- o INSPECCION
```

7.2 Salida de Prueba_2

Prueba_2 imprime tres tablas de generadores antes de la simulación, con formato:

```
=====
RESULTADOS GENERADOR LLEGADAS
=====
No.      Ri              Uniforme(4,10)
1         0.34521         6.07126
2         0.89034         9.34204
...

```

Las columnas para INSPECCION muestran Exponencial(5) y para EMPAQUE muestran Exponencial(8). Al finalizar la simulación imprime:

```
=====
RESULTADOS DE SIMULACION
=====
Piezas llegadas: NNN
Piezas inspeccionadas: NNN
Piezas empacadas: NNN
Piezas perdidas: NNN
Piezas pendientes: NNN

Cola inspección: N
Cola empaque: N

Utilización inspector: XX.XX%
Utilización empacadores: XX.XX%

Utilidad generada: $XXX.XX

Cuello de botella: EMPAQUE    <- o INSPECCION

```

7.3 Calculo de Métricas

Métrica	Formula	Descripción
Utilización inspector	$\text{tiempoOcupadoInspector} / \text{TIEMPO_SIMULACION} * 100$	Porcentaje del tiempo total que el inspector estuvo procesando piezas.
Utilización empacador	$\text{tiempoOcupadoEmpacador} / \text{TIEMPO_SIMULACION} * 100$	Porcentaje del tiempo total que el empacador estuvo activo.
Utilidad económica	$\text{piezasEmpacadas} * 0.40$	Ingreso total generado a razón de \$0.40 por pieza empacada exitosamente.
Cuello de botella	$\text{utilEmpacador} > \text{utilInspector} ?$ EMPAQUE : INSPECCION	El recurso con mayor utilización es el cuello de botella del sistema.

8. Instalación y Ejecución

8.1 Requisitos del Sistema

Requisito	Valor
Sistema Operativo	Windows 10 o Windows 11 (64 bits)
Arquitectura	x86-64 (AMD64)
Java en el sistema	No requerido (JVM embebida OpenJDK 25.0.1)
Espacio en disco	Aprox. 250 MB por versión
Memoria RAM	Mínimo 256 MB libres para la JVM
Permisos	Lectura/escritura en la carpeta de instalación

8.2 Descarga del programa (Repositorio de Github)

Descargar el archivo .zip completo con ambos programas o bajarlos por separado según la necesidad desde la siguiente liga:

https://github.com/Daelin-lin/Pruebas_Simulacion

8.3 Pasos de Instalación

Opción 1: Release de GitHub

1. Ingresar al repositorio del proyecto.
2. Abrir la sección Releases.
3. Descargar el archivo ZIP más reciente.
4. Extraer el contenido en una carpeta local.

Opción 2: Repositorio Completo

1. Ingresar al repositorio.
2. Seleccionar Code.
3. Elegir Download ZIP.
4. Extraer el archivo descargado.
5. Abrir la carpeta Ejecutables.
6. Navegar a la versión requerida:
Main o Pruebas_2
7. Ejecutar Main.exe o Prueba_2.exe desde su carpeta correspondiente con doble clic o desde una ventana de comandos.
8. Ingresar los parámetros del LGC cuando la consola lo solicite (tres veces, una por generador).

Importante: No mover el .exe fuera de su carpeta. El launcher busca el runtime en la subcarpeta runtime/ relativa a su ubicación.

9. Generador Lineal Congruencial

9.1 Condiciones para Periodo Completo (Hull-Dobell)

Para que un Generador Congruencial Lineal (LGC) produzca la secuencia más larga posible antes de repetir valores (es decir, que tenga período máximo), deben cumplirse tres condiciones:

1. El incremento c y el módulo m no deben tener divisores comunes distintos de 1, es decir, deben ser coprimos.
2. Si un número primo divide al módulo m , también debe dividir a $(a-1)$. Esto asegura que la secuencia pueda recorrer adecuadamente todos los valores posibles.
3. Si el módulo m es divisible entre 4, entonces $(a-1)$ también debe ser divisible entre 4. Esta condición adicional garantiza el período máximo cuando el módulo contiene el factor 4.

Cuando estas tres condiciones se cumplen simultáneamente, el generador puede recorrer todos los valores posibles antes de comenzar a repetir la secuencia, obteniendo así un período máximo igual a m .

Nota: El programa no valida estas condiciones. Una mala elección de parámetros puede producir periodos cortos y resultados estadísticamente incorrectos.

9.2 Parámetros Recomendados

1.

Parámetro	Valor sugerido
a (multiplicador)	57
c (incremento)	1021
x0 (semilla)	7
m (modulo)	4096
N	5000 o más

2.

Parámetro	Valor sugerido
a (multiplicador)	153
c (incremento)	125
x0 (semilla)	100
m (modulo)	4096
n	5000 o más

3.

Parámetro	Valor sugerido
a (multiplicador)	12869
c (incremento)	13849
x0 (semilla)	4513
m (modulo)	4096
n	5000 o más

Reproducibilidad: *Para reproducir exactamente los mismos resultados en dos ejecuciones distintas, todos los parámetros (a, c, x0, m, n) deben ser idénticos en las tres llamadas al generador.*

10. Solución de Problemas

Sintoma	Causa probable	Solucion
El .exe no inicia o cierra inmediatamente	La carpeta runtime/ no esta junto al ejecutable.	Descomprimir el ZIP completo. No mover el .exe de forma aislada.
Error: could not find or load main class	El .cfg apunta a una clase incorrecta.	Verificar que app.mainclass sea Main.Main o Prueba_2.Prueba_2.
Ventana cierra antes de leer resultados	Se presiono ENTER antes de revisar.	Ejecutar desde cmd.exe o redirigir salida a archivo de texto.
Resultados distintos en dos ejecuciones	Semilla (x0) diferente.	Usar el mismo x0 en todas las ejecuciones que se quieran comparar.
Piezas perdidas siempre 0 en Main	Main no tiene limite explicito en colaEmpaque.	Comportamiento esperado. Solo Prueba_2 descarta piezas cuando colaEmpaque ≥ 5 .
Utilización > 100%	n muy pequeno; los RI se reciclan y acumulan tiempo incorrecto.	Usar un n mayor o igual al número esperado de eventos en la simulación.
NumberFormatException al ingresar parámetros	Se ingreso un valor no numérico o con coma decimal.	Usar solo dígitos enteros. El separador decimal no aplica; los parámetros son tipo long/int.
Cuello de botella siempre EMPAQUE	La media de empaque (8 min) es mayor que la de inspección (5 min), lo que es esperable.	Comportamiento normal. Para cambiar el cuello de botella habría que modificar las medias en el código fuente.

11. Runtime Java Embebido

El runtime incluido fue generado con jlink a partir de OpenJDK 25.0.1. Contiene los módulos necesarios para ejecutar aplicaciones de escritorio Java completas. Los módulos incluidos son:

java.base, java.compiler, java.datatransfer, java.desktop, java.instrument, java.logging, java.management, java.management.rmi, java.naming, java.net.http, java.prefs, java.rmi, java.scripting, java.security.jgss, java.security.sasl, java.smartcardio, java.sql, java.sql.rowset, java.transaction.xa, java.xml, java.xml.crypto, jdk.accessibility, jdk.attach, jdk.compiler, jdk.dynalink, jdk.httpserver, jdk.incubator.vector, jdk.internal.ed, jdk.internal.jvmstat, jdk.internal.le, jdk.internal.md, jdk.internal.opt, jdk.jartool, jdk.javadoc, jdk.jconsole, jdk.jdwp.agent, jdk.jdi, jdk.jfr, jdk.jshell, jdk.jsobject, jdk.management, jdk.management.agent, jdk.management.jfr, jdk.net, jdk.nio.mapmode, jdk.sctp, jdk.security.auth, jdk.security.jgss, jdk.unsupported, jdk.unsupported.desktop, jdk.xml.dom, jdk.zipfs.

Observación: La aplicación solo requiere *java.base* (Scanner, Math, LinkedList, Queue). El conjunto de módulos incluido es mucho más amplio porque JPackage incluye por defecto el JDK completo a menos que se especifique explícitamente con `--add-modules`. En una versión optimizada, el runtime podría reducirse a menos de 20 MB usando: `jpackage --add-modules java.base`.

12. Glosario

Termino	Definición
SED (Simulación de Eventos Discretos)	Técnica de modelado en la que el estado del sistema cambia únicamente en instantes puntuales (eventos), no de forma continua.
LGC (Generador Lineal Congruencial)	Algoritmo determinístico para producir secuencias de números pseudoaleatorios mediante la recurrencia $X(n+1) = (a \cdot X(n) + c) \bmod m$.
RI (Numero Pseudoaleatorio)	Valor en $[0, 1)$ producido por el LGC. Se usa como entrada para las funciones de distribución inversa.
Transformada Inversa	Método para generar muestras de una distribución de probabilidad F usando un RI uniforme: aplica $F^{-1}(RI)$.
Cuello de Botella	Recurso del sistema con la mayor utilización, que limita el rendimiento global del proceso.
Periodo del LGC	Longitud de la secuencia antes de que se repita. Un LGC con periodo completo genera m valores distintos antes de ciclar.
JPackage	Herramienta del JDK que empaqueta una aplicación Java con su propio runtime en un instalador o ejecutable nativo por plataforma.
jlink	Herramienta del JDK que genera un runtime Java personalizado incluyendo solo los módulos necesarios.
Cola FIFO	Estructura de datos en la que el primer elemento en entrar es el primero en salir (First In, First Out).

13. Soporte técnico

Para contactar a soporte técnico puede enviar un correo a:

- Herrera Martínez Aylin
herrera.martinez.aylin23@gmail.com
- Gutiérrez Rivas Alejandro
ale.gtzrivas@gmail.com
- Vázquez Alviso Bryan Alexei
bavazqueza@gmail.com

--- Fin del Manual Técnico ---